

Databases in AFFiNE

This document provides a comprehensive overview of how databases are implemented in AFFiNE, covering storage mechanisms, visualization views, and potential future enhancements.

Table of Contents

1. Overview
2. Internal Storage Mechanisms
 - Web Storage (IndexedDB)
 - Native Storage (SQLite)
 - In-Memory Database Models
3. Database Model Structure
4. Property Types
 - Basic Types
 - Special Types
5. Database Visualization Views
 - Table View
 - Kanban View
 - Connection Between Data Source and Views
6. Potential Enhancements
 - Adding Unique Identifier Data Type
 - Adding Relations Between Tables
 - Creating Merged Views from Multiple Tables
 - Adding Horizontal Table View
 - Adding Computational Output Property Type
 - Adding Spreadsheet Functionality
7. Implementation Details

Overview

AFFiNE's database feature allows users to organize and visualize structured data in different formats. The implementation employs a flexible architecture with a clear separation between data models, storage, and visualization.

Key components of the database system include:

1. **Data Source:** The core data provider that manages access to database records and properties
2. **Storage Engines:** Platform-specific implementations for persisting data
3. **View System:** Visualization components that render data in different formats (tables, kanban boards)
4. **Property Types:** Various data types for storing different kinds of information

Internal Storage Mechanisms

AFFiNE uses different storage mechanisms depending on the platform it's running on:

Web Storage (IndexedDB)

For web-based installations, AFFiNE uses IndexedDB, a low-level browser API for client-side storage of significant amounts of structured data.

Key implementations: - IndexedDBDocStorage in `/packages/common/nbstore/src/impls/idb/doc.ts`
- IndexedDBBlobStorage in `/packages/common/nbstore/src/impls/idb/blob.ts`
- IndexedDBIndexerStorage in `/packages/common/nbstore/src/impls/idb/indexer/index.ts`

The IndexedDB schema includes tables for: - **snapshots**: Document snapshots - **updates**: Document updates - **clocks**: Document timestamps - **blobs**: Binary data objects - **blobData**: Actual binary data content - **indexerRecords**: For full-text search and indexing - **invertedIndex**: For efficiently querying indexed content

```
export interface DocStorageSchema extends DBSchema {
  snapshots: {
    key: string;
    value: {
      docId: string;
      bin: Uint8Array;
      createdAt: Date;
      updatedAt: Date;
    };
    // ...other tables...
  };
  // ...more tables...
}
```

Native Storage (SQLite)

For desktop/mobile applications, AFFiNE uses SQLite for data persistence.

Key implementations: - SqliteDocStorage in `/packages/frontend/native/nbstore/src/storage.rs`
- SqliteBlobStorage in `/packages/common/nbstore/src/impls/sqlite/blob.ts`

The SQLite schema includes tables similar to the IndexedDB structure but implemented in SQL:

```
// Schema excerpt from /packages/frontend/native/schema/src/v1.rs
pub const SCHEMA: &str = r#"CREATE TABLE IF NOT EXISTS "updates" (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  data BLOB NOT NULL,
  timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP NOT NULL,
  doc_id TEXT
```

```
);
// ...other tables...
"#;
```

In-Memory Database Models

Beyond persistent storage, AFFiNE implements efficient in-memory structures for database operations:

1. **Addressable Data Structure:**
 - Each database model maintains an in-memory representation using Y.js structures
 - Data is addressable through unique identifiers for each row and property
 - The structure follows a schema: `database → rows → cells`
2. **Data Retrieval Methods:**
 - Direct access to cells via row and property identifiers: `database.getRow(rowId).getCell(property)`
 - Query-based access using filters and selectors
 - Events subscription for reactive data updates
3. **Memory Management:**
 - Lazy loading of row data for large databases
 - Memory usage optimization through virtual scrolling
 - Serialization/deserialization between memory and storage formats
4. **Technical Implementation:**
 - Accessible via `DatabaseModel` instances in `/blocksuite/affine/model/src/blocks/database/dat`
 - Memory structure maps to Y.js shared types (`Y.Map`, `Y.Array`)
 - Transaction-based updates for data consistency

This architecture allows applications to efficiently address and retrieve database content with minimal storage/memory overhead.

Database Model Structure

AFFiNE's database structure is based on a flexible model with the following key components:

1. **Database Block:** The container for the entire database
 - Implemented in `/blocksuite/affine/model/src/blocks/database/database-model.ts`
2. **Properties:** Define data columns and their types
 - Property types are defined in `/blocksuite/affine/blocks/database/src/properties/index.ts` and property-specific files
3. **Rows:** Individual records in the database
 - Rows can be added, deleted, and reordered
4. **Cells:** Individual data points at the intersection of rows and properties
 - Cells store typed data according to their property type
5. **Views:** Different visualizations of the same data (table, kanban)
 - Views can have filters, sorts, and grouping

The core data source implementation for database blocks is in: -
`/blocksuite/affine/blocks/database/src/data-source.ts`

Property Types

AFFiNE's database system supports various property types for storing different kinds of data:

1. Basic Types

- **Title**: Special property type that represents the main identifier for a row
- **Rich Text**: For formatted text content
- **Number**: For numeric values with formatting options
- **Checkbox**: For boolean values (true/false)
- **Date**: For date and time values
- **Select**: For single selection from predefined options
- **Multi-select**: For multiple selections from predefined options
- **Progress**: For percentage values (0-100%)

2. Special Types

- **Link**: For references to other documents or URLs
- **Image**: For embedding images
- **File**: For attaching files
- **Created Time**: Automatically generated timestamp

Property types are implemented as property models with consistent interfaces:

```
export type PropertyModel<
  Type extends string = string,
  PropertyData extends Record<string, unknown> = Record<string, unknown>,
  RawValue = unknown,
  JsonValue = unknown,
> = {
  type: Type;
  config: PropertyConfig<PropertyData, RawValue, JsonValue>;
  create: Create<PropertyData>;
  createPropertyMeta: CreatePropertyMeta<
    Type,
    PropertyData,
    RawValue,
    JsonValue
  >;
};
```

Each property type defines: - How to store and validate data - How to render and edit the property in the UI - How to convert between different formats (string, JSON, raw value) - Default values and behavior

Database Visualization Views

AFFiNE currently supports two main visualization views for databases:

Table View

The table view presents data in a traditional spreadsheet-like format with rows and columns.

Key components: - `TableSingleView` in `/blocksuite/affine/data-view/src/view-presets/table/table-view.ts`
- `TableViewTable` in `/blocksuite/affine/data-view/src/view-presets/table/pc/table-view.ts`
- `VirtualTableView` in `/blocksuite/affine/data-view/src/view-presets/table/pc-virtual/table-view.ts`
(for large tables) - `MobileDataViewTable` in `/blocksuite/affine/data-view/src/view-presets/table/mobile/table-view.ts`

Features: - Column resizing and reordering - Row filtering and sorting - Grouping data by property values - Cell editing and formatting - Statistical calculations

Kanban View

The kanban view visualizes data as cards organized into columns, typically grouped by a select-type property.

Key components: - `KanbanSingleView` in `/blocksuite/affine/data-view/src/view-presets/kanban/kanban-view.ts`
- `DataViewKanban` in `/blocksuite/affine/data-view/src/view-presets/kanban/pc/kanban-view.ts`
- `MobileDataViewKanban` in `/blocksuite/affine/data-view/src/view-presets/kanban/mobile/kanban-view.ts`

Features: - Cards grouped by property values - Drag-and-drop card movement between groups - Card details display with properties - Add new groups and cards

Connection Between Data Source and Views

The architecture separates the data model from its visualization:

1. **DataSource**
 - `DataSourceBase` in `/blocksuite/affine/data-view/src/core/data-source/base.ts`
 - `DatabaseBlockDataSource` in `/blocksuite/affine/blocks/database/src/data-source.ts`
2. **ViewManager**
 - `ViewManager` in `/blocksuite/affine/data-view/src/core/view-manager/view-manager.ts`
 - `SingleViewBase` in `/blocksuite/affine/data-view/src/core/view-manager/single-view.ts`
3. **View Implementation**
 - `DataViewBase` in `/blocksuite/affine/data-view/src/core/view/data-view-base.ts`

This separation allows: - The same data to be visualized in different ways - Consistent data operations across views - View-specific operations (like filtering and sorting) that don't affect the underlying data

Potential Enhancements

Adding Unique Identifier Data Type

Currently, AFFiNE databases don't have a dedicated unique identifier data type. Adding this feature would enable:

1. **Auto-incrementing IDs:**
 - Automatically assign sequential numbers or custom prefixed IDs to new rows
 - Implementation would require a new property type extending the existing property model architecture
2. **Custom Formats with Prefixes/Suffixes:**
 - Allow format specifications like "PRJ-{number}" or "{number}-REF"
 - Would require a formatter system similar to the number property's formatter
3. **Technical Approach:**
 - Create a new property type in `/blocksuite/affine/data-view/src/property-presets/`
 - Implement ID generation and validation logic
 - Add UI components for configuring ID format
 - Update the database model to handle unique constraints

Adding Relations Between Tables

AFFiNE currently lacks direct relations between database tables. Adding this feature would enable:

1. **One-to-many and Many-to-many Relations:**
 - Link records across different database blocks
 - Create parent-child relationships between records
2. **Relation Property Type:**
 - Display selected related records
 - Allow quick navigation between related records
 - Filter and sort based on related record properties
3. **Technical Approach:**
 - Create a new relation property type
 - Implement a system to reference other database blocks
 - Add UI for selecting related records
 - Implement logic for maintaining referential integrity
4. **Table Derivation:**
 - Define template tables that can be used as blueprints
 - Derive multiple specialized tables from a single template
 - Maintain schema inheritance relationships between parent and derived tables
 - Enable easy aggregation of derived tables that share the same template
 - Support overriding specific properties while inheriting others

Creating Merged Views from Multiple Tables

AFFiNE currently only shows data from one database at a time. Adding merged views would enable:

1. **Combined Data Visualization:**
 - Show data from multiple database blocks in a single view
 - Apply filters and sorts across combined data
2. **Join Operations:**
 - Define how data from different sources should be combined
 - Support inner joins, left joins, etc. based on matching property values
3. **Technical Approach:**
 - Create a new merged-view data source that combines multiple data sources
 - Implement join logic for matching records
 - Update the view system to handle data from multiple sources
 - Add UI for configuring merged views
4. **Template-Based Aggregation:**
 - Automatically identify and aggregate tables derived from the same template
 - Apply consistent operations across all derived tables
 - Provide unified views that respect the derivation hierarchy
 - Enable template-level property changes to propagate to all derived tables
 - Support filtering derivation relationships in the aggregated view

Adding Horizontal Table View

A horizontal table view would transpose the traditional table visualization, making properties appear as rows and records as columns:

1. **Key Features:**
 - Properties displayed as row headers
 - Records displayed as column headers
 - Transposed view of the same data
 - Enhanced record comparison capabilities
2. **User Interface:**
 - Row headers displaying property names
 - Column headers displaying record titles
 - Cell editing and formatting consistent with standard table view
 - Filtering and sorting applied across the transposed dimension
 - Support for horizontal scrolling with fixed property names
3. **Technical Approach:**
 - Create specialized view components:
 - `HorizontalTableSingleView`
 - `TableViewHorizontalTable`
 - `MobileTableViewHorizontalTable`

- Implement transpose adapter for existing data source
 - Optimize for efficient rendering of wide tables
 - Support all existing filtering and sorting operations in the transposed context
4. **Use Cases:**
 - Comparing multiple records side-by-side
 - Presenting databases with many properties and few records
 - Analyzing differences between similar items
 - Creating record comparison reports

Adding Computational Output Property Type

Adding a computational output property type would allow for dynamic values derived from other properties:

1. **Script-Based Calculation:**
 - Support for both JavaScript and Python scripting languages
 - Short command syntax via @ (e.g., @sum(price*quantity))
 - Complex data manipulation capabilities
2. **JavaScript Implementation:**
 - Access to row and database data via API
 - Methods like `data.from(...).where(...).get(...)`
 - Special keyword `current` to reference current row
 - Example:

```
@sum(price * quantity)
```

```
// More complex script
data.from("inventory")
  .where("category", "electronics")
  .where("price", "<", current.budget)
  .get("items")
  .length
```
3. **Python Implementation:**
 - Integration with Python runtime (via WebAssembly or native backend)
 - Access to pandas-like data manipulation for databases
 - Example:

```
@sum(df['price'] * df['quantity'])
```

```
# More complex calculation
df = data.from_table("inventory")
filtered = df[(df.category == "electronics") &
              (df.price < current.budget)]
len(filtered.items)
```
4. **Technical Approach:**
 - Create a new property type with script evaluation capabilities

- Implement sandboxed execution environment for scripts
- Provide standard libraries for data manipulation
- Add caching for computed results to improve performance
- Support reactive updates when dependent properties change

Adding Spreadsheet Functionality

While AFFiNE’s current database implementation focuses on structured data with predefined property types, adding true spreadsheet functionality would provide more flexible data manipulation capabilities:

1. **Cell-based Calculation Engine:**
 - Implement a formula evaluation engine that supports common spreadsheet functions (SUM, AVERAGE, IF, etc.)
 - Enable cell references using coordinates (e.g., A1, B2) or named ranges
 - Support cross-sheet references for complex calculations
2. **Grid-based UI with Free-form Data:**
 - Allow any cell to contain any type of data regardless of column
 - Support merging cells and custom formatting
 - Implement cell-level permissions and protection
3. **Enhanced Data Analysis:**
 - Add pivot tables for dynamic data summarization
 - Implement conditional formatting based on cell values or formulas
 - Add charting capabilities for data visualization
4. **Technical Approach:**
 - Create a new block type specifically for spreadsheets, separate from the database blocks
 - Implement a grid-based data model with its own storage format
 - Develop a formula parser and evaluation engine
 - Design a specialized UI for spreadsheet interaction (formula bar, sheet tabs, etc.)
 - Consider compatibility with standard spreadsheet formats (CSV, XLSX) for import/export
5. **Integration with Databases:**
 - Allow linking spreadsheet ranges to database properties
 - Enable syncing between spreadsheet views and database views of the same data
 - Support using database data as inputs to spreadsheet calculations

Implementation Details

The database feature in AFFiNE is built on several architectural layers:

1. **Storage Layer**
 - Handles persistence across platforms (IndexedDB for web, SQLite for native)
2. **Data Layer**

- Manages data structures and operations
 - Mediates between storage and visualization
3. **View Layer**
 - Renders data in different formats
 - Handles user interactions and events
 4. **Property System**
 - Defines different data types and behaviors
 - Manages data validation and conversion

This modular architecture allows for future extensions while maintaining compatibility with existing features.